

Generating-Function and Holonomic-Recurrence Methods for Constrained Motzkin Paths with Prescribed Zero-Level Contacts - CS215 Discrete Math (H) Project

Abstract

We study a lattice-path enumeration problem that can be formulated as follows: count the number of length- n walks on $\mathbb{Z}_{\geq 0}$ starting at height 0, using step set $\{+1, 0, -1\}$, never going below 0, ending at height 0, and visiting height 0 exactly k times among the $n + 1$ vertices (including the start and end). This family refines Motzkin paths by the number of contacts with the horizontal axis. We derive an exact coefficient-extraction formula via a return-block decomposition and show that the desired count equals

$$[x^n] \left(\frac{1 + x - \sqrt{1 - 2x - 3x^2}}{2} \right)^{k-1}.$$

To support very large parameter regimes under modular arithmetic (e.g., n up to 10^7 modulo an arbitrary prime), we develop a second-order linear differential equation for the generating function and convert it into a low-memory, linear-time recurrence for coefficients. This yields a practical, modulus-agnostic alternative to polynomial square-root / power methods that rely on NTT-friendly primes.

1. Problem formulation

A **Motzkin path** of length n is a sequence of heights $(h_0, h_1, \dots, h_n)$ such that:

1. $h_0 = 0, h_i \in \mathbb{Z}_{\geq 0}$ for all i .
2. For each step $i \rightarrow i + 1$, we have $h_{i+1} - h_i \in \{+1, 0, -1\}$.
3. (Return condition) $h_n = 0$.

Let $A_{n,k}$ be the number of such paths with **exactly k indices** $i \in \{0, 1, \dots, n\}$ satisfying $h_i = 0$. Equivalently, the path touches the axis at exactly k vertices.

We aim to characterize $A_{n,k}$ both combinatorially (closed-form coefficient extraction) and algorithmically (fast computation modulo a prime).

2. Decomposition by zero-level return blocks

2.1. Primitive return blocks

Consider any Motzkin path that starts at height 0 and ends at height 0. Examine the segment between two **consecutive** axis-touching vertices. By definition of “consecutive,” the path does **not** touch height 0 in between. Such a segment is a **primitive return block**. There are exactly two structural types:

- **Type H (horizontal on the axis):** a single step $(1, 0)$ taken while at height 0. This creates an immediate new axis touch at the next vertex.
- **Type U-...-D (arch):** the block begins with an up-step $(1, +1)$, ends with a down-step $(1, -1)$, and the interior stays at height ≥ 1 . Shifting the interior down by 1 yields an ordinary Motzkin path (possibly empty).

Let $M(x) = \sum_{n \geq 0} M_n x^n$ be the ordinary generating function for Motzkin paths of length n (from height 0 to 0, never below 0). The standard first-return decomposition gives the classical functional equation

$$M(x) = 1 + xM(x) + x^2M(x)^2,$$

corresponding to: empty path (1), horizontal step at level 0 then a Motzkin path (xM), or an arch $U(\text{Motzkin})D$ followed by a Motzkin path (x^2M^2).

Now define $u(x)$ as the generating function of a **single primitive return block** (nonempty return from axis to axis with no intermediate axis contact). By the two block types above:

$$u(x) = x + x^2M(x).$$

- The x corresponds to Type H (one horizontal step on the axis).
- The $x^2M(x)$ corresponds to Type U-...-D: one up step + one down step + an arbitrary Motzkin interior (shifted), hence length $2 + \ell$.

Using the quadratic equation for M , one may eliminate M and obtain a quadratic equation for u directly (derived in §4):

$$u^2 - (1 + x)u + (x + x^2) = 0,$$

so

$$u(x) = \frac{1 + x - \sqrt{1 - 2x - 3x^2}}{2},$$

where the sign is chosen so that $u(0) = 0$ (formal power series condition).

2.2. Exactly k axis contacts $\iff$ concatenation of $k - 1$ blocks

A path with exactly k axis-touching vertices has exactly $k - 1$ consecutive gaps between them, hence it is a concatenation of exactly $k - 1$ primitive return blocks. Conversely, concatenating $k - 1$ primitive return blocks produces a nonnegative Motzkin path that touches the axis exactly at block boundaries, i.e., exactly k times.

Therefore the ordinary generating function for paths with exactly k axis contacts is simply

$$F_k(x) = u(x)^{k-1}.$$

Extracting coefficients gives the central identity:

Proposition 2.1 (Coefficient formula).

For $n \geq 0$ and $k \geq 1$,

$$A_{n,k} = [x^n](u(x)^{k-1}) = [x^n] \left(\frac{1+x-\sqrt{1-2x-3x^2}}{2} \right)^{k-1}.$$

This converts the constrained path count into coefficient extraction of an algebraic power series.

3. DP viewpoint and generating-function compression

A dynamic-programming formulation mirrors the block decomposition. Let B_n be the number of primitive return blocks of length n , i.e. $u(x) = \sum_{n \geq 1} B_n x^n$. Let $A_{n,k}$ be as above.

Concatenation of $k-1$ blocks with total length n yields the convolution:

$$A_{n,k} = \sum_{\substack{n_1 + \dots + n_{k-1} = n \\ n_i \geq 1}} B_{n_1} \cdots B_{n_{k-1}}.$$

This is exactly the coefficient identity $A_{n,k} = [x^n]u(x)^{k-1}$. Thus the generating function not only provides a closed form; it compresses a k -fold convolution into a single algebraic series.

4. Algebraic specification of $u(x)$ and implicit differentiation

Let

$$u(x) = \frac{1+x-\sqrt{1-2x-3x^2}}{2}.$$

It is algebraic of degree 2. Define the polynomial

$$P(u, x) = u^2 - (1+x)u + (x+x^2).$$

Then $P(u(x), x) = 0$ identically.

4.1. First derivative $u'(x)$

Differentiate implicitly:

$$0 = \frac{d}{dx} P(u(x), x) = P_u(u, x) \cdot u'(x) + P_x(u, x),$$

so

$$u'(x) = -\frac{P_x(u, x)}{P_u(u, x)}.$$

Compute partial derivatives:

$$P_u(u, x) = 2u - (1 + x), \quad P_x(u, x) = -u + (1 + 2x).$$

Hence

$$u' = \frac{u - (1 + 2x)}{2u - (1 + x)}.$$

To rationalize the denominator, use the identity (from the closed form)

$$(2u - (1 + x))^2 = 1 - 2x - 3x^2.$$

Thus

$$\frac{1}{2u - (1 + x)} = \frac{2u - (1 + x)}{1 - 2x - 3x^2},$$

and therefore

$$u' = \frac{(u - (1 + 2x))(2u - (1 + x))}{1 - 2x - 3x^2}.$$

Eliminate u^2 via $u^2 = (1 + x)u - (x + x^2)$ (i.e., $P(u, x) = 0$), yielding

$$u' = \frac{-(1 + 3x)u + (1 + x)}{1 - 2x - 3x^2}.$$

This “linear-in- u ” representation is key to eliminating algebraic quantities later.

5. From algebraic series to a holonomic differential equation for $F(x) = u(x)^m$

Let $m = k - 1$ and

$$F(x) = u(x)^m.$$

Since algebraic functions are D-finite (holonomic) and D-finiteness is preserved under powering, F satisfies a linear differential equation with polynomial coefficients.

5.1. Expressing F'/F and F''/F in a two-dimensional basis

By logarithmic differentiation,

$$\frac{F'}{F} = m \frac{u'}{u}.$$

From the expression for u' ,

$$\frac{u'}{u} = \frac{-(1+3x) + \frac{1+x}{u}}{1-2x-3x^2}.$$

Use $P(u, x) = 0$ to express $1/u$ as a linear function of u . Indeed,

$$u^2 - (1+x)u + x(1+x) = 0 \implies u(u - (1+x)) = -x(1+x),$$

so

$$\frac{1}{u} = \frac{1+x-u}{x(1+x)}.$$

Substituting gives

$$\frac{u'}{u} = \frac{-u+1-3x^2}{x(1-2x-3x^2)}.$$

Hence

$$F' = mF \cdot \frac{-u+1-3x^2}{x(1-2x-3x^2)}.$$

Differentiating again expresses F'' as a combination of F and F' with coefficients that are rational functions in x and linear in u . Because the field extension $\mathbb{Q}(x) \subset \mathbb{Q}(x, u)$ has degree 2, every such expression lies in a two-dimensional $\mathbb{Q}(x)$ -vector space with basis $\{1, u\}$. Consequently, there exists a nontrivial $\mathbb{Q}(x)$ -linear combination of F, F', F'' that cancels the u -component, yielding a differential equation over $\mathbb{Q}(x)$.

5.2. Explicit second-order ODE

Proposition 5.1 (Holonomic differential equation).

Let $m \geq 0$ and $F(x) = u(x)^m$ with u defined by $P(u, x) = 0$ and $u(0) = 0$. Then F satisfies

$$A(x)F''(x) + B(x)F'(x) + C(x)F(x) = 0,$$

where

$$A(x) = x(1+x)^2(1-3x),$$

$$B(x) = (1+x)\left((1-3x-6x^2) - m(1-x-6x^2)\right),$$

$$C(x) = mx\left((4+3x) - m(2+3x)\right).$$

Equivalently, expanding polynomials:

$$(-3x^4 - 5x^3 - x^2 + x)F'' + ((6m-6)x^3 + (7m-9)x^2 - 2x + (1-m))F' + ((3m-3m^2)x^2 + (4m-2m^2)x)F = 0.$$

This ODE is the analytical engine behind a fast coefficient recurrence.

6. Converting the ODE into a coefficient recurrence

Write

$$F(x) = \sum_{n \geq 0} a_n x^n, \quad a_n = A_{n,k}.$$

Then

$$F'(x) = \sum_{n \geq 0} (n+1) a_{n+1} x^n, \quad F''(x) = \sum_{n \geq 0} (n+2)(n+1) a_{n+2} x^n.$$

Multiplying by the polynomials $A(x), B(x), C(x)$ and equating coefficients of x^n yields a finite-width recurrence. After simplification, one obtains a convenient three-step forward recurrence.

Proposition 6.1 (Linear-time recurrence for a_n).

Fix $m \geq 0$ and let $a_n = [x^n]u(x)^m$. For $n \geq 2$,

$$(n+1)(n+1-m) a_{n+1} = n(n+1) a_n + (m-n+1)(2m-5n+1) a_{n-1} + 3(m-n+1)(m-n+2) a_{n-2}.$$

This recurrence determines all coefficients from three initial values.

6.1. Initial conditions and low-degree structure

Because $u(x) = x + x^2 + x^3 + 2x^4 + \dots$, we have:

- If $m = 0$, then $F(x) = 1$ so $a_0 = 1$ and $a_n = 0$ for $n \geq 1$.
- If $m \geq 1$, then $u(x)^m$ has lowest degree m , hence $a_n = 0$ for $n < m$ and $a_m = 1$.

Moreover, writing $u(x) = x(1 + x + x^2 + 2x^3 + \dots)$ gives

$$a_{m+1} = m, \quad a_{m+2} = \frac{m(m+1)}{2}.$$

From these, the recurrence computes $a_{m+3}, a_{m+4}, \dots, a_n$.

6.2. Modular arithmetic and the “any prime” regime

Suppose we want $a_n \bmod p$ for a prime p (e.g., $p = 10^9 + 7$). The recurrence involves division by $(n+1)(n+1-m)$. Over $\mathbb{F}_p$, division is multiplication by modular inverses:

$$t^{-1} \equiv t^{p-2} \pmod{p} \quad (t \not\equiv 0 \pmod{p}).$$

For typical ranges like $n \leq 10^7$ and $p \approx 10^9$, $n+1 \not\equiv 0 \pmod{p}$. The factor $n+1-m$ becomes 0 only at $n = m-1$, i.e., at a single small index; this is harmless because the first nonzero coefficient is a_m , which is already known, and one simply starts the forward recurrence at $n = m+1$ using (a_m, a_{m+1}, a_{m+2}) .

Hence the coefficient computation is linear time $O(n)$ and works for any prime modulus, including non-NTT primes like $10^9 + 7$.

7. Algorithmic discussion

7.1. Polynomial square root + exponentiation (moderate scale)

From

$$A_{n,k} = [x^n] \left(\frac{1 + x - \sqrt{1 - 2x - 3x^2}}{2} \right)^m,$$

a direct approach is:

1. Compute the truncated series $\sqrt{1 - 2x - 3x^2}$ to degree n .
2. Form $u(x)$ to degree n .
3. Compute $u(x)^m \bmod x^{n+1}$ via fast exponentiation with polynomial multiplication.

With NTT-friendly primes (such as 998244353), steps (1) and (3) can be carried out in $\tilde{O}(n)$ time using Newton iteration and NTT multiplication. This is effective up to about 10^5 – 10^6 depending on constants and memory. For very large n (e.g., 10^7) or moduli like $10^9 + 7$, the need for CRT/multi-mod NTT or slower multiplication makes this approach less attractive.

7.2. Holonomic recurrence (large scale, any prime)

The recurrence in Proposition 6.1 avoids polynomial multiplication entirely. It needs only:

- $O(1)$ arithmetic per coefficient,
- a few running values (a_t, a_{t-1}, a_{t-2}) ,
- modular inverses of linear factors (straightforward when p is prime and $n \ll p$).

Thus, for n on the order of 10^7 , the holonomic recurrence is typically the simplest and fastest approach in practice, and it is modulus-agnostic.

8. Conclusion

By refining Motzkin paths according to the number of zero-level contacts, we obtained a decomposition into primitive return blocks. This yields the coefficient formula

$$A_{n,k} = [x^n] u(x)^{k-1}, \quad u(x) = \frac{1 + x - \sqrt{1 - 2x - 3x^2}}{2}.$$

To support extreme parameter sizes under modular arithmetic—especially for primes like $10^9 + 7$ where NTT-based power-series algebra is inconvenient—we derived a second-order linear differential equation for $F(x) = u(x)^m$ and converted it into a three-step forward recurrence for coefficients. The resulting method computes $A_{n,k}$ in $O(n)$ time and $O(1)$ memory, and works under any prime modulus.

Appendix. C++ Implementation

```
#define _USE_MATH_DEFINES
#include <bits/stdc++.h>

#define PI M_PI
#define E M_E
#define npt nullptr
#define SON i->to
#define OPNEW void* operator new(size_t)
#define ROPNEW void* Edge::operator new(size_t){static Edge* P = ed; return P++;}
#define ROPNEW_NODE void* Node::operator new(size_t){static Node* P = nd; return P++;}

using namespace std;

mt19937 rnd(random_device{}());
int rndd(int l, int r){return rnd() % (r - l + 1) + l;}
bool rnddd(int x){return rndd(1, 100) <= x;}

typedef unsigned int uint;
typedef unsigned long long unll;
typedef long long ll;
typedef long double ld;

#define MOD (99824435311)

template < typename T = int >
inline T read(void);

int N, K;
ll dp[11000000];
ll inv[11000000];

int main(){
    N = read(), K = read();
    if(K > N)printf("0\n"), exit(0);
    inv[1] = 1;
    for(int i = 2; i <= N + 100; ++i)inv[i] = (MOD - MOD / i) * inv[MOD % i] % MOD;
    dp[0] = 1;
    dp[1] = K - 1;
    dp[2] = (ll)K * (K - 1) / 2 % MOD;
    for(int i = 0; i + 3 <= N - K + 1; ++i)
        dp[i + 3] = (
            dp[i] * 3 * i % MOD * (i + 1) % MOD +
            dp[i + 1] * ((5 * i + 3 * K + 6) % MOD) % MOD * (i + 1) % MOD +
            dp[i + 2] * (i + K + 1) % MOD * (i + K + 2) % MOD
        ) % MOD * inv[i + 3] % MOD * inv[i + K + 2] % MOD;
    printf("%lld\n", dp[N - K + 1]);
    fprintf(stderr, "Time: %.6lf\n", (double)clock() / CLOCKS_PER_SEC);
    return 0;
}
```

```

template < typename T >
inline T read(void){
    T ret(0);
    int flag(1);
    char c = getchar();
    while(c != '-' && !isdigit(c))c = getchar();
    if(c == '-')flag = -1, c = getchar();
    while(isdigit(c)){
        ret *= 10;
        ret += int(c - '0');
        c = getchar();
    }
    ret *= flag;
    return ret;
}

```

References

1. Flajolet, P., & Sedgewick, R. (2009). *Analytic Combinatorics*. Cambridge University Press.

- Publisher page (Cambridge Core): <https://www.cambridge.org/core/books/analytic-combinatorics/7E37474C43E9B95C90BEDE082CF28708>
- DOI: <https://doi.org/10.1017/CBO9780511801655>
- Free author site / book page (Princeton): <https://ac.cs.princeton.edu/home/>
- Free PDF (Princeton): <https://ac.cs.princeton.edu/AC.pdf>
- Author mirror PDF (INRIA): <https://algo.inria.fr/flajolet/Publications/book.pdf>

2. Stanley, R. P. (1999). *Enumerative Combinatorics, Volume 2*. Cambridge University Press. (Cambridge Studies in Advanced Mathematics, 62).

- Publisher page (Cambridge Core): <https://www.cambridge.org/core/books/enumerative-combinatorics/D8DDDDFF7E8EBF0BCFE99F5E6918CE2A8>
- DOI: <https://doi.org/10.1017/CBO9780511609589>
- Author page (MIT): <https://math.mit.edu/~rstan/ec/>
- Table of contents (MIT): <https://math.mit.edu/~rstan/ec/ec2toc.html>
- Google Books preview: https://books.google.com/books/about/Enumerative_Combinatorics_Volume_2.html?id=zg5wDqT6T-UC

3. Petkovšek, M., Wilf, H. S., & Zeilberger, D. (1996). *A = B*. A K Peters/CRC Press.

- Publisher page (Taylor & Francis): <https://www.taylorfrancis.com/books/mono/10.1201/9781439864500/-marko-petkovsek-herbert-wilf-doron-zeilberger>
- DOI: <https://doi.org/10.1201/9781439864500>
- Free author page (UPenn): <https://www2.math.upenn.edu/~wilf/AeqB.html>
- Free full-book PDF (UPenn): <https://www2.math.upenn.edu/~wilf/AeqB.pdf>
- Free full-book PDF (Rutgers mirror): <https://www.math.rutgers.edu/~zeilberg/AeqB.pdf>

4. Chyzak, F. (2000). An extension of Zeilberger's fast algorithm to general holonomic functions. *Discrete Mathematics*, 217(1–3), 115–134.
 - DOI: [https://doi.org/10.1016/S0012-365X\(99\)00259-9](https://doi.org/10.1016/S0012-365X(99)00259-9)
 - Official open-archive page (ScienceDirect): <https://www.sciencedirect.com/science/article/pii/S0012365X99002599>
 - Author PDF (INRIA): <https://specfun.inria.fr/chyzak/Publications/Chyzak-2000-EZF.pdf>
 - ScienceDirect PDF entry point: <https://www.sciencedirect.com/science/article/pii/S0012365X99002599/pdf>
5. von zur Gathen, J., & Gerhard, J. (2013). *Modern Computer Algebra* (3rd ed.). Cambridge University Press.
 - Publisher page (Cambridge Core): <https://www.cambridge.org/core/books/modern-computer-algebra/DB3563D4013401734851CF683D2F03F0>
 - DOI: <https://doi.org/10.1017/CBO9781139856065>
 - Google Books preview: https://books.google.com/books/about/Modern_Computer_Algebra.html?id=AE5PN5QGgvUC
6. Wilf, H. S. (2005). *generatingfunctionology* (3rd ed.). A K Peters/CRC Press.
 - Publisher page (Taylor & Francis): <https://www.taylorfrancis.com/books/mono/10.1201/b10576/generatingfunctionology-herbert-wilf>
 - DOI (publisher DOI form): <https://doi.org/10.1201/b10576>
 - Publisher preview PDF (PagePlace): https://api.pageplace.de/preview/DT0400.9781439864395_A24326158/preview-9781439864395_A24326158.pdf
 - Free earlier-edition PDF by author (useful open reference copy): <https://www2.math.upenn.edu/~wilf/gfology2.pdf>
7. Motzkin number. (2025). In *Wikipedia*. Retrieved January 10, 2026.
 - Permanent revision (stable “oldid”): https://en.wikipedia.org/w/index.php?title=Motzkin_number&oldid=1321323601
 - Main page: https://en.wikipedia.org/wiki/Motzkin_number
 - Cross-check reference (MathWorld): <https://mathworld.wolfram.com/MotzkinNumber.html>
8. OEIS Foundation Inc. (n.d.). *The On-Line Encyclopedia of Integer Sequences*: A001006 (Motzkin numbers). Retrieved January 10, 2026.
 - Main entry: <https://oeis.org/A001006>
 - Internal-format view: <https://oeis.org/A001006/internal>
 - B-file (data): <https://oeis.org/A001006/b001006.txt>
9. Bostan, A., & Salvy, B., et al. (selected works on fast recurrence/D-finite computation).
 - 9.1. Bostan, A., Gaudry, P., & Schost, É. (2007). Linear recurrences with polynomial coefficients and computation of the n th term of a sequence. *SIAM Journal on Computing*, 36(6), 1777–1806.
 - SIAM DOI landing: <https://epubs.siam.org/doi/10.1137/S0097539704443793>
 - DOI: <https://doi.org/10.1137/S0097539704443793>
 - Author PDF (Salvy): <https://perso.ens-lyon.fr/bruno.salvy/BostanGaudrySchost.pdf>
 - 9.2. Bostan, A., Chyzak, F., Lecerf, G., Salvy, B., & Schost, É. (2007). Differential equations for algebraic functions. In *Proceedings of ISSAC 2007* (pp. 25–32). ACM.
 - ACM DOI landing: <https://dl.acm.org/doi/10.1145/1277548.1277553>
 - DOI: <https://doi.org/10.1145/1277548.1277553>

- arXiv preprint: <https://arxiv.org/abs/cs/0703121>
- Author PDF (Bostan page): <https://mathexp.eu/bostan/publications/BoChLeSaSc07.pdf>
- Author PDF (UW Waterloo mirror): <https://cs.uwaterloo.ca/~eschost/publications/issac75-bostan.pdf>